Sequential Optimization for AutoEncoder Training (Fixed Model/Loss/Oracle)

Our contribution: **implement AdaGrad + Lion** and design **scheduling + hyperparameters** to minimize **training loss**.

Moneer Ghanem, Karam Qawasmi

Convex Optimization Course

Update Formulas

- **AdaGrad**: $$g_t = \nabla f(\theta_t), \quad G_t = G_{t-1} + g_t \odot g_t, \quad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{G_t + \epsilon}}$$
  - Stabilizes directions with consistently large gradients (shrinks their steps).
  - **Failure mode**: Gt grows monotonically → effective LR shrinks → stagnation late.

- **Lion:** $$m_t = \beta m_{t-1} + (1-\beta) g_t, \quad \theta_{t+1} = \theta_t - \eta \cdot \mathrm{sign}(m_t)$$
  - Uses direction (sign) rather than magnitude → robust refinement when gradients get tiny/noisy.
  - **Failure mode:** if theta too large → overshoot/oscillation (step doesn't naturally shrink with gradient magnitude).

Key Hypothesis (Why Sequential)

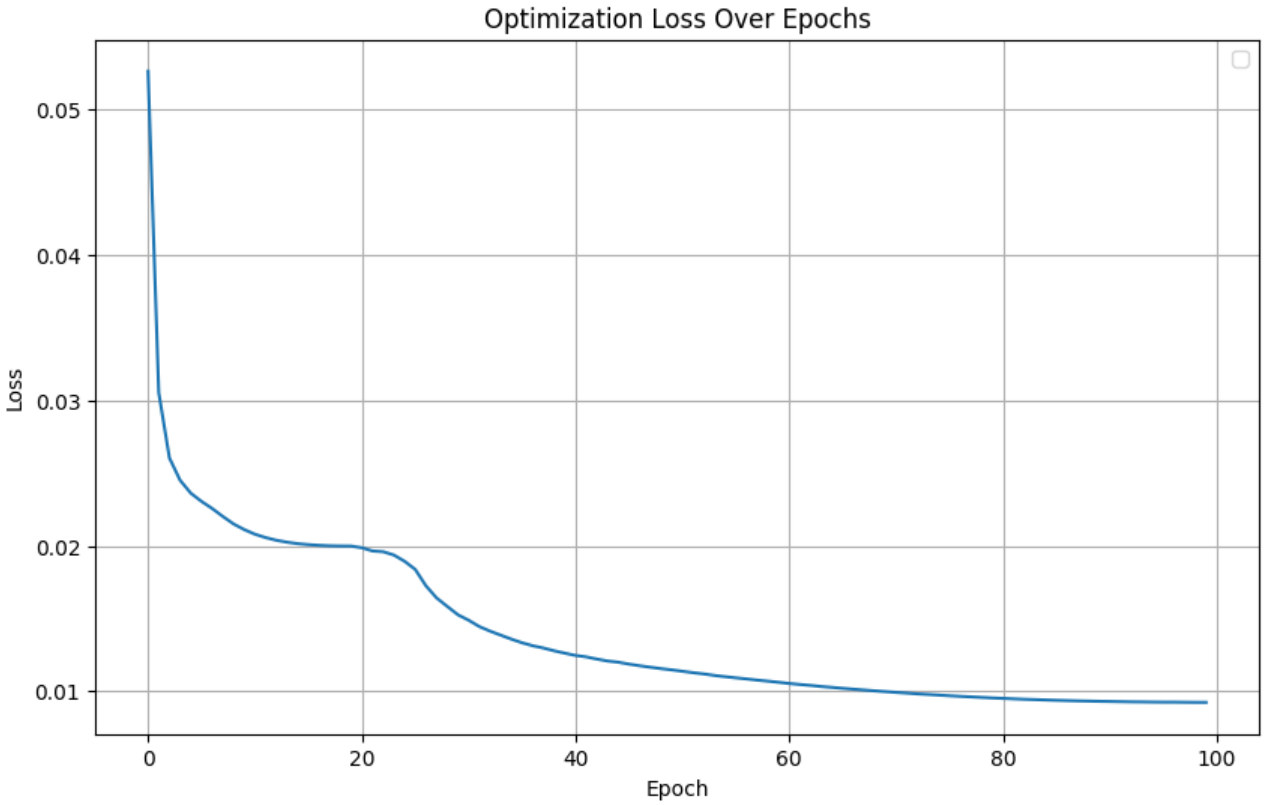
- One optimizer is not best across all training stages
- **AdaGrad**: fast + stable early, but can **stall late** (effective LR collapses).
- **Lion**: strong late refinement, but can be **unstable early** (sign steps + noisy gradients).
- **Hypothesis:** AdaGrad for *basin finding* → Lion for *refinement*
  - **Phase 1 ("basin finding")**: AdaGrad quickly reduces loss while accumulator is still small.
  - **Phase 2 ("refinement")**: Lion continues improving when magnitudes shrink and AdaGrad stalls.

Final Training Recipe

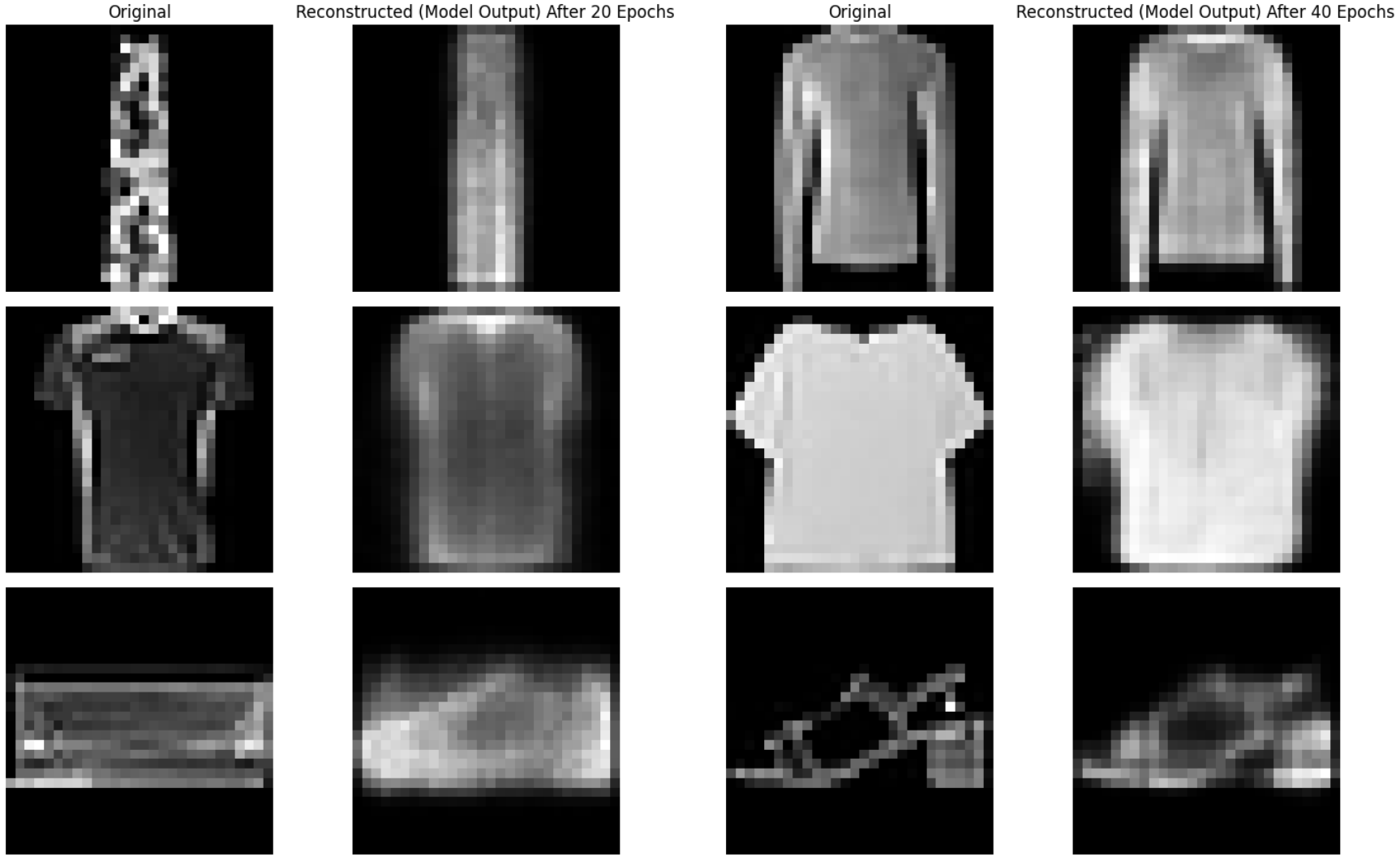
- **Hybrid schedule:** 20% AdaGrad → 80% Lion (100 epochs)
- **Learning rates:** AdaGrad 1.5e-2, Lion 1e-3
- **LR schedule (both phases): Warmup → Cosine Annealing**
  - AdaGrad phase: 4-epoch **linear warmup → cosine decay**
  - Lion phase**:** 7-epoch **linear warmup (0.1× → 1× LR)** → **cosine decay** to **η_min = 1e−5**
- **Batch size: 350**
- **Weight decay:** 0 (pure training-loss objective)
- **Why this works?**
  - AdaGrad early: accumulator not yet large → steps aren't crushed → fast progress.
  - Lion late: directional sign updates keep moving when gradients get small.
  - Warmup prevents unstable early steps (cold-start momentum / abrupt step-size change), making the switch smoother and improving final loss.
  - Cosine decay: smooth annealing → reduces late oscillations and supports settling into a **lower-loss basin**.

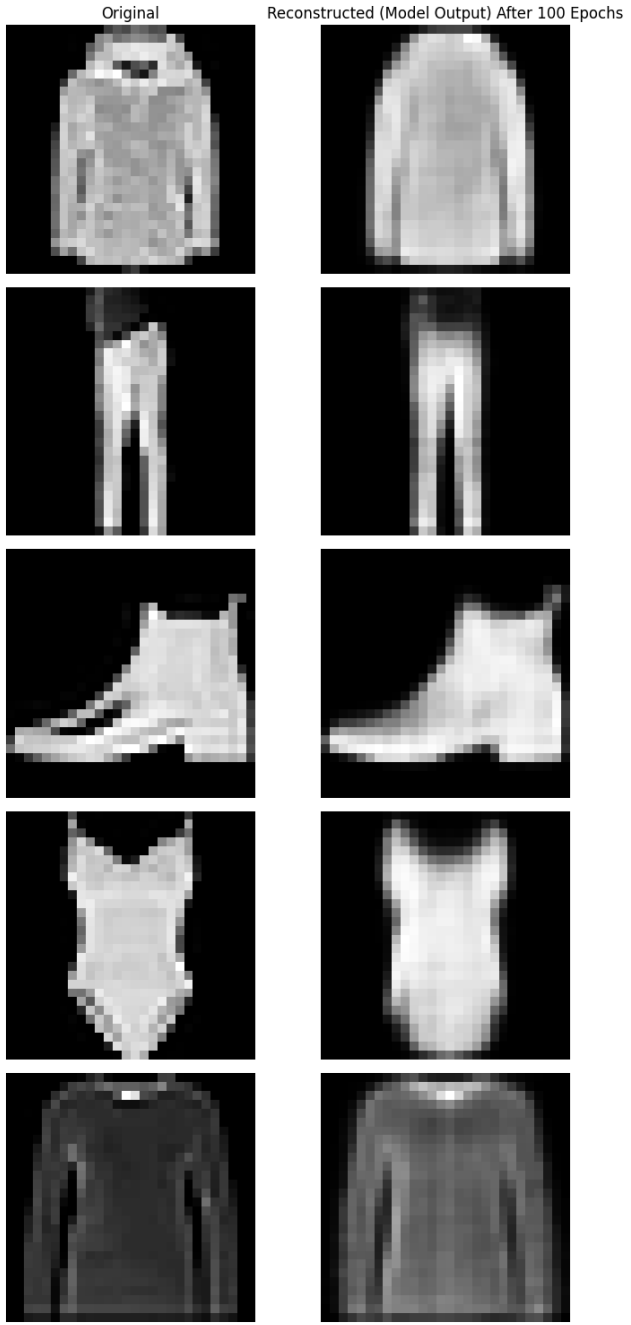
Main Result: Loss Curve With Switch Point

- Final loss: **0.008573**
- Runtime: about **7 minutes** using Colab's **CPU**
- Improvement: **85.44%**
- Observation: 'knee' around switch → Lion continues descent
- Warmup + cosine improved final loss by **~10.5%** vs the previous best (**0.009584 → 0.008573**).



Reconstructions Examples





Comparison Summary + Core Failure Modes

- **AdaGrad only:** strong start → slow late improvement
- **Lion only:** can reach **0.102,** still worse than hybrid in same number of epochs
- Hybrid wins by using complementary strengths
- Theory:
  - AdaGrad: growth → effective LR → ~0 → slow late convergence
  - Lion: too-large η → overshoot/oscillation (sign step doesn't shrink near minima)

Exploration: What Failed & Why

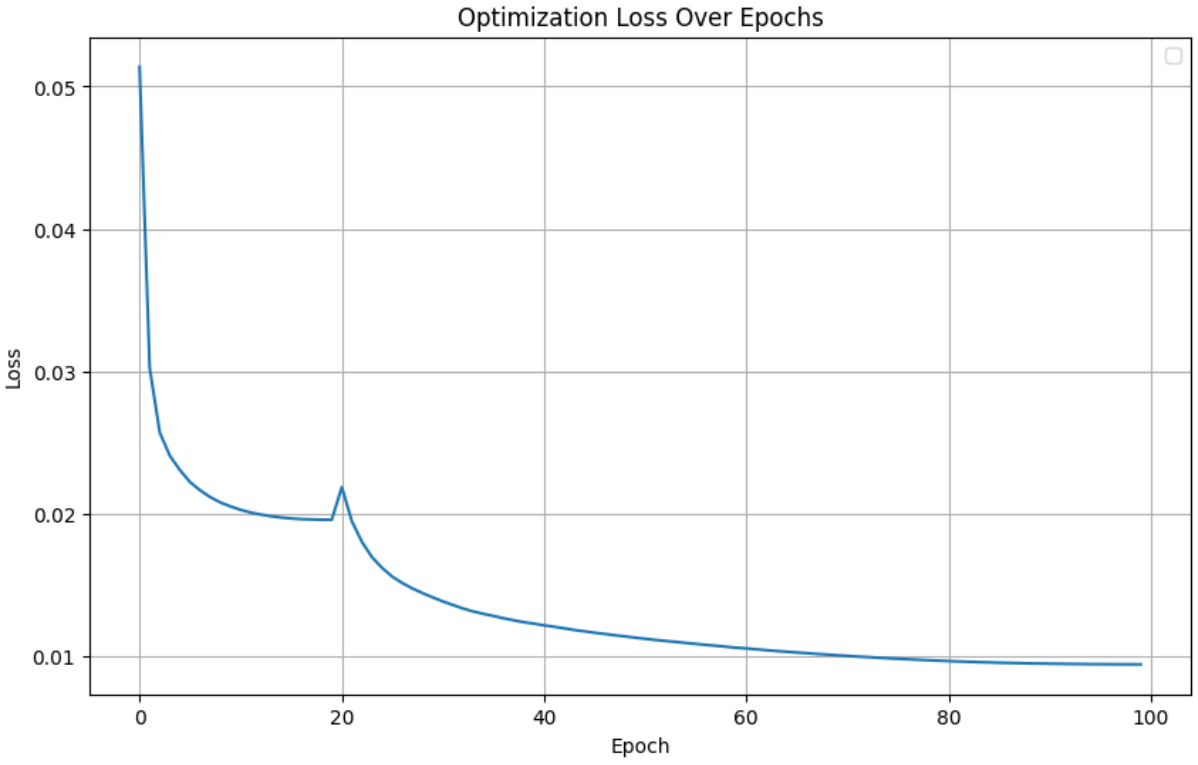
1) Learning-rate instability

- **Small LR:** very slow convergence (tiny steps relative to curvature/scale)
- **Large LR:** loss spikes / oscillation
- **Theory (Lion):** updates have magnitude ~η per coordinate via sign; near minima steps don't automatically shrink → too-large η overshoots.

Exploration: What Failed & Why

2) Warmup choice

- **Without warmup:** early-2nd phase instability → worse final loss. (0.0095)
- **With warmup:** gradual LR ramp stabilizes updates (momentum "cold start" + sign steps), then cosine annealing handles refinement.



Exploration: What Failed & Why

3) Batch size sensitivity (e.g., 64 vs 350)

- Observation: larger batch gave smoother Lion behavior and better results.
- **Theory:** small batches → high gradient variance → sign can flip randomly → random-walk-like updates; larger batches stabilize direction, making sign updates reliable.

Exploration: What Failed & Why

4) Single-optimizer limitations

- **Lion only:** reached **~0.103** in ~100 epochs, but underperformed the hybrid.
- **AdaGrad at the end:** significantly worse late-stage progress.
- **Theory:** denominator $\sqrt{\sum g^2}$ becomes large → effective LR collapses → near-stagnation.

5) Successful final change

AdaGrad early + Lion late exploited complementary strengths → best loss.

Conclusions & Future Work

- **Key insights**
  - Optimizer effectiveness is **phase-dependent**.
  - Sequential optimization can outperform single-optimizer baselines.
  - Scheduling + batch size matter especially for sign-based methods.
  - **Warmup is crucial when switching optimizers** (reduces transient instability and improves the final basin reached).
- **Future work**
  - Adaptive switch criterion (loss-plateau detection / gradient-norm trigger) instead of fixed 20%.
  - More principled batch-size/LR scaling analysis (compare by steps or wall-clock, not just epochs).
  - Explore other methods that include hessian or an approximation

Summary

In this project, we investigated sequential optimization strategies for training an AutoEncoder on Fashion-MNIST under a fixed model, loss, and oracle framework. Our central insight is that optimizer effectiveness is phase-dependent: magnitude-adaptive methods such as AdaGrad enable rapid and stable early descent ("basin finding") but tend to stagnate later as the accumulated squared-gradient history grows and the effective learning rate collapses, whereas sign-based methods like Lion provide effective late-stage refinement but can be unstable during initialization if the learning rate is too large. By combining these optimizers in a sequential scheme—switching from AdaGrad to Lion after 20% of the total epochs—and using a **warmup → cosine annealing** learning-rate schedule in both phases, we achieved a final training loss of **0.008573** (**85.44%** improvement over the initial loss). These results demonstrate that sequential strategies can significantly outperform single-optimizer baselines by leveraging complementary algorithmic strengths, and that **warmup is particularly important when switching to Lion**, as it reduces transient instability and improves the final basin reached. Future work would explore adaptive switching criteria based on loss-plateau detection rather than fixed epoch thresholds.